Airwallex Agentic Banking Hackathon · Starter Kit 11

Purser

Your AI agents spend through Airwallex and Metal.
The CFO stays in control.

Each agent gets an Airwallex company card and a Metal on-chain mandate, within limits the CFO sets. When an agent needs more, it stops and asks, and the CFO approves on a phone with Face ID.

Airwallex Metal with Claude

The problem

Agents can pay now. Who holds the limit?

- An agent with a card or a wallet spends as fast as it is told to, including by a fake invoice or a planted instruction.
- A limit written in a prompt can be talked out of. It has to live in the rails and in code.
- The CFO needs to approve the exact terms, not a summary, and to prove later what was approved.

> **$3,046,598,558 in business email compromise losses were reported by victims to the FBI in 2025.**
>
> FBI IC3, 2025 Internet Crime Report

How it works

Alone inside the limits. Face ID for the rest.

The agent may, alone

Pay inside its card limits and its on-chain mandate.

The CFO approves

Anything above a limit, a new payee, a bank-detail change, a looser policy: the exact terms, signed with Face ID on the phone. The runner checks the signature and the terms' hash before money moves.

Exceptions

A price rise is refused on chain and becomes options. A suspected fake invoice or a planted instruction waits for a person. An agent that pays too often is stopped.

How we use Airwallex

Cards, FX and payouts, with Airwallex enforcing the limits

- **A virtual Visa card per agent** (Issuing): limits from the CFO policy, merchant categories, currency and end date fixed per agent; all written to the card and read back.
- **Declined by Airwallex itself** in the recorded run: €45.00 over the limit (`LIMIT_EXCEEDED`), €19.99 at the wrong kind of shop (`MERCHANT_CATEGORY_NOT_ALLOWED`). After the CFO's Face ID raise, the same charge went through.
- **Single-use task cards** for a flight, a hotel or a train, from one sentence (separate live run).
- **A €1,850.00 invoice by SWIFT**: only the EUR shortfall (after the reserve and the demo plan's set-aside) bought on a locked quote; one Face ID covers rate, payee and fee ceiling; the approval's hash rides in the transfer's metadata.
- **Global Accounts** (separate live run): retailers' payments matched to invoices; anything off goes to the CFO.

How we use Metal

Mandates on chain: Metal holds the limit, not only Purser

- **A mandate per agent**: a cap per payment, a total budget, one payee, an end date.
- **Metal's facilitator refused a price rise** against the mandate on chain (`mandate_cap_exceeded`): nothing moved. The CFO signed new terms with Face ID; the mandates were replaced and the old one is refused.
- **x402 with signed orders** for per-request sales, **mandate pulls** for refunds and the sweep, **pay-per-call clearing** for a prepaying customer (separate live run).
- **Earnings are booked at Airwallex only when final**, exactly once (a simulated sandbox deposit that names the testnet transaction).
- **The Dispute desk** (after the run): the simulated customer disputed 12 orders; each was checked against the chain with the signed order Northwind keeps; the claim refunded nothing (orders 7, 10, 13 had been refunded earlier for wrong data).

For a bigger company

More control moves on chain

- **Four eyes on chain**: a 2-of-2 Metal account (CFO + controller). One signature, or the CFO with a stranger's key, reverted on chain; both owners together passed.
- **Compliance before a payout**: a send-policy contract pays only with Purser's signed screening decision. Eleven attacks reverted on chain. Mandate payments do not pass through it.
- **Finality certificates** kept as evidence for each payment seen within about 2 hours (the validators' signature is not verified: their testnet keys are not published).
- **Clearing under load**: 2,000 of 2,000 payments at 4.24 a second, about $0.0007 of RPC per payment (an estimate).
- **ISO 20022**: the paid invoice as one pacs.008 message linking the SWIFT transfer and the Metal transaction.
- All in separate test worlds on Metal testnet, 30 October; not in the recorded run.

Where Claude is used

Claude reads and advises. Code and the CFO decide.

Haiku

Sorts each incoming document.

Sonnet

Reads invoices and notices into fields that code then checks.

Opus

Reviews a proposal's risk for the CFO; played the attacker in the red team.

Claude holds no key and approves nothing. In the recorded run: 23 calls, $0.093 per €1,000 the agents paid.

Proof

Every payment has a proof

Friday run 1, 30 October 2026, with a Dispute desk case afterwards

106
payments inside the limits, no approval at the time (about half the simulated customer's)

8
stops by the card, the chain, the fraud checks or the CFO

€3,719.99
kept out for good

4
decisions signed with Face ID

- Red team (scripted attacks plus 4 documents by Claude as the attacker, against in-memory stand-ins): 39 of 39 failed, 0 inconclusive; 37 stopped before succeeding, 2 found afterwards, not prevented.
- Scenario Lab: 18 of 18 stories as expected. Audit log: 194 rows, hash-chained.
- All test money: the Airwallex sandbox and Metal testnet. Simulated: merchants' charges, deposits, a transfer's last steps; the customer Summit and the warehouse API Stockline are run by us.

Try it

npm ci && npm run judge

No keys, no network after install.

- **/** what each rail did, with its proof
- **/?recorded** the CFO dashboard at the end of the run
- **/metal** every Metal feature with its proof
- **/live** the working day as an animation on the two rails
- **/lab** 18 stories, from a normal purchase to the emergency stop

Built with Claude Code. Sources: docs/claims.md.